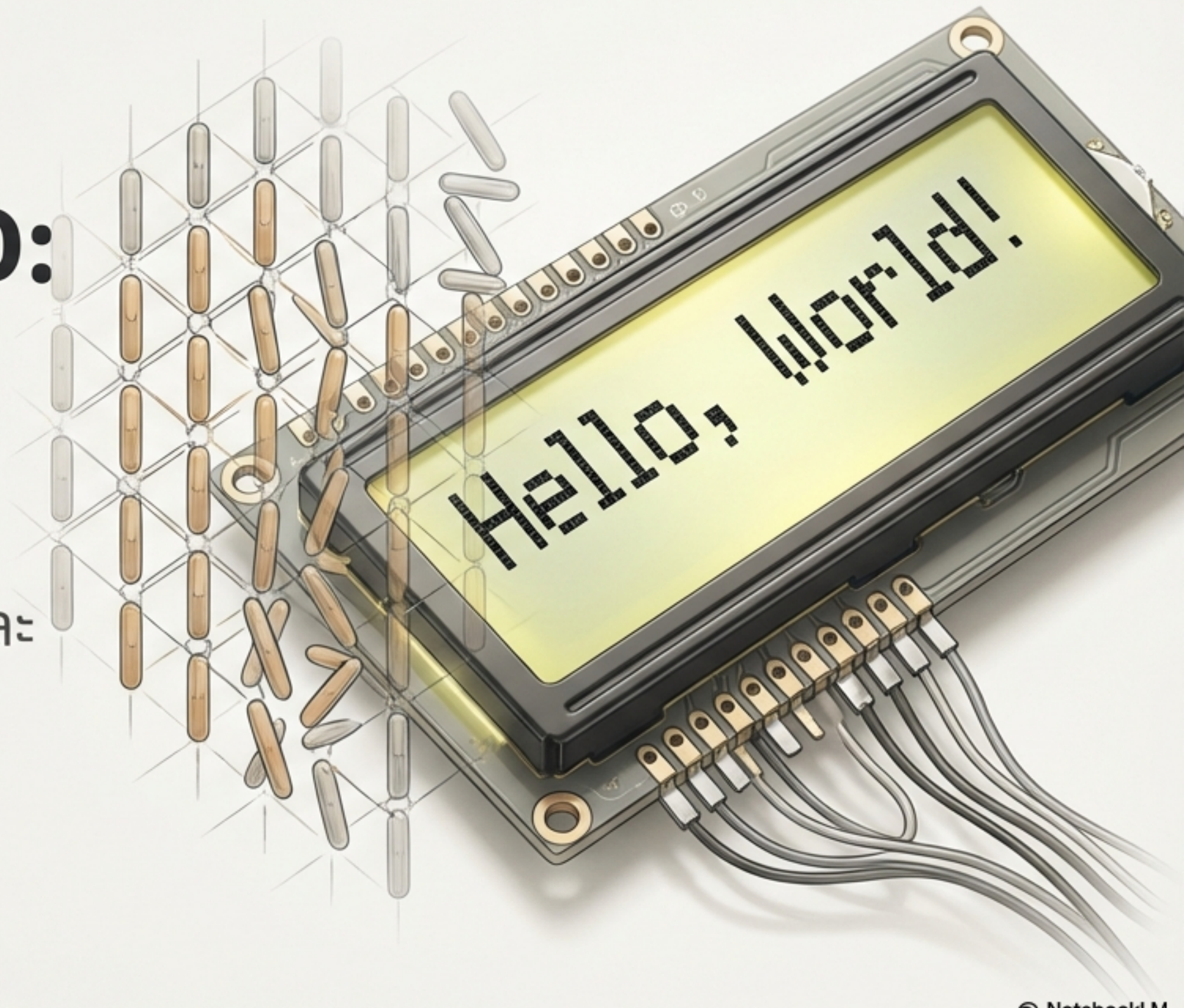


ถอดรหัสหน้าจอ LCD: จากโมเลกุลสู่โค้ด ESP32 ของคุณ

คู่มือฉบับสมบูรณ์เพื่อทำความเข้าใจและ
ใช้งานจอ LiquidCrystal_I2C



ทำไมต้องชื่อ ‘Liquid Crystal’?

ชื่อของไลบรารีไม่ได้มาแบบสุ่ม แต่บอกถึงหัวใจของเทคโนโลยีที่อยู่ข้างใน



`Liquid`

ของเหลว



`Crystal`

ผลึก



`Liquid Crystal`

สารที่มีคุณสมบัติ
‘กึ่งของเหลว กึ่งผลึก’

สารที่ไหลได้ แต่จัดเรียงตัวเป็นระเบียบ



เหมือนของเหลว

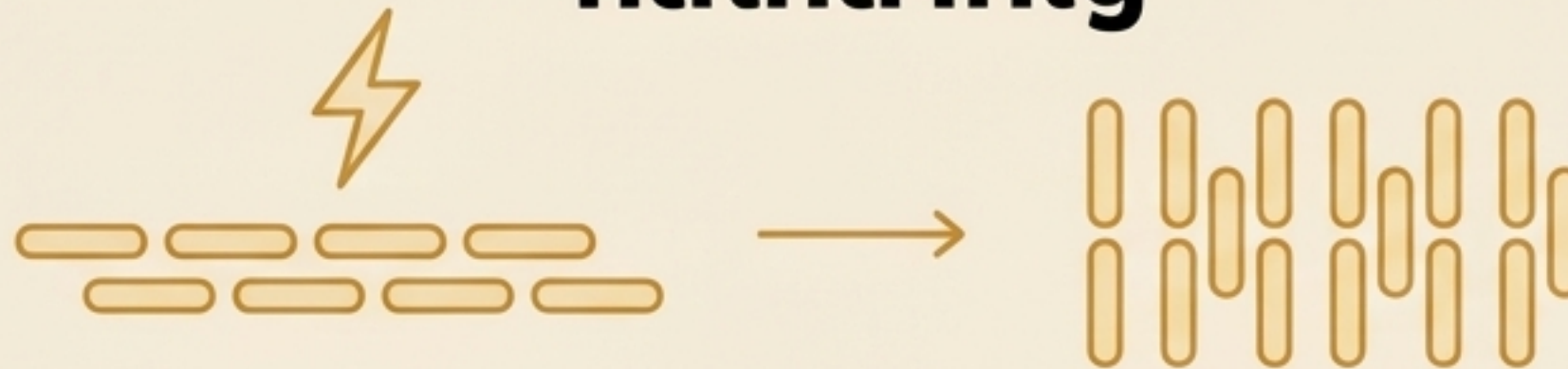
โมเลกุลสามารถไหลและเคลื่อนที่ได้



เหมือนผลึก

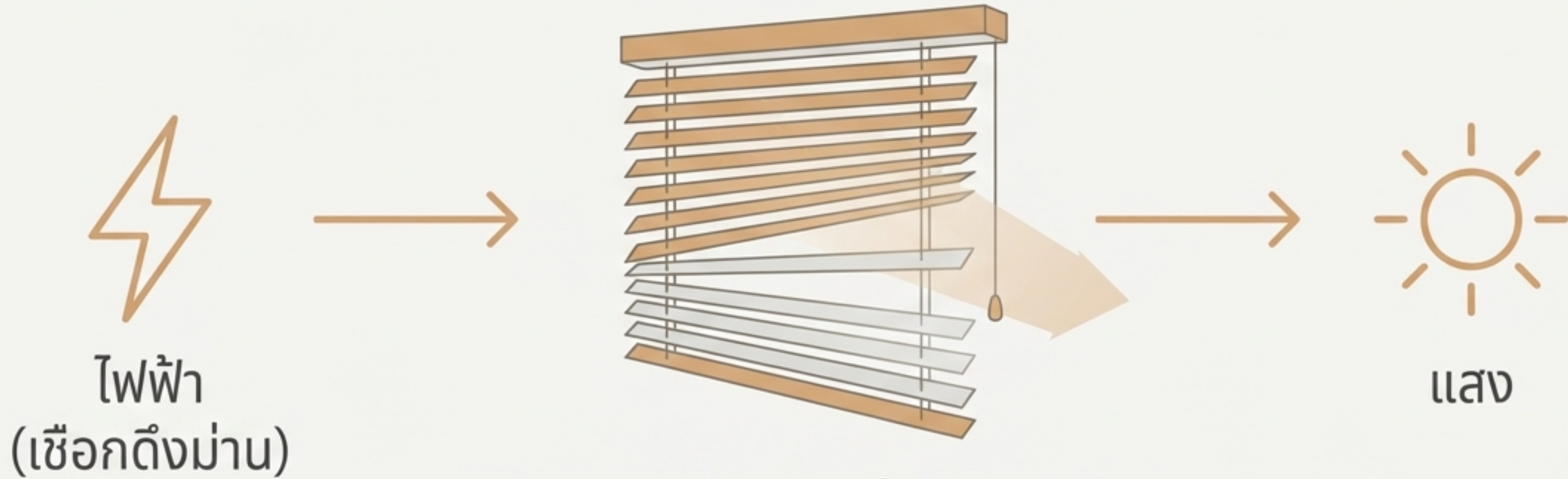
แต่การเรียงตัวของโมเลกุลนั้นมีระเบียบ

กลไกสำคัญ



โมเลกุล Liquid Crystal สามารถ **เปลี่ยนการเรียงตัว**เมื่อได้รับไฟฟ้า
นี่คือจุดที่ทำให้เราใช้มันควบคุมการเดินทางของแสงได้

ลองนึกภาพ Liquid Crystal เป็น "ม่านปรับแสง"

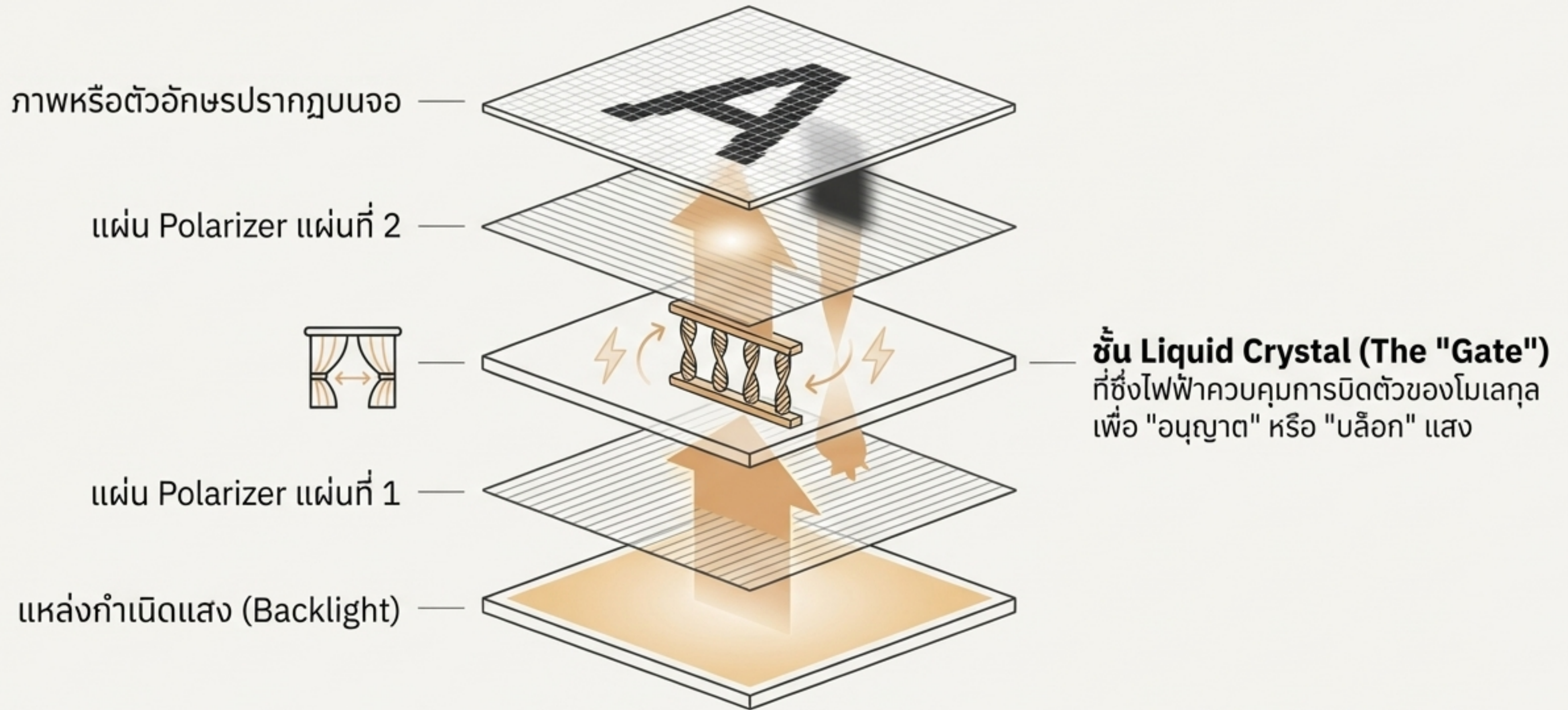


Liquid Crystal (ม่านปรับแสง)

- **ไฟฟ้ามาก** → ม่านปิดตัวมาก → **แสงผ่านน้อย**
- **ไฟฟ้าน้อย** → ม่านปิดตัวน้อย → **แสงผ่านมาก**

ตัวอักษรบนจอก็คือการเปิด-ปิดม่านเหล่านี้เป็นล้านๆ จุดนั่นเอง

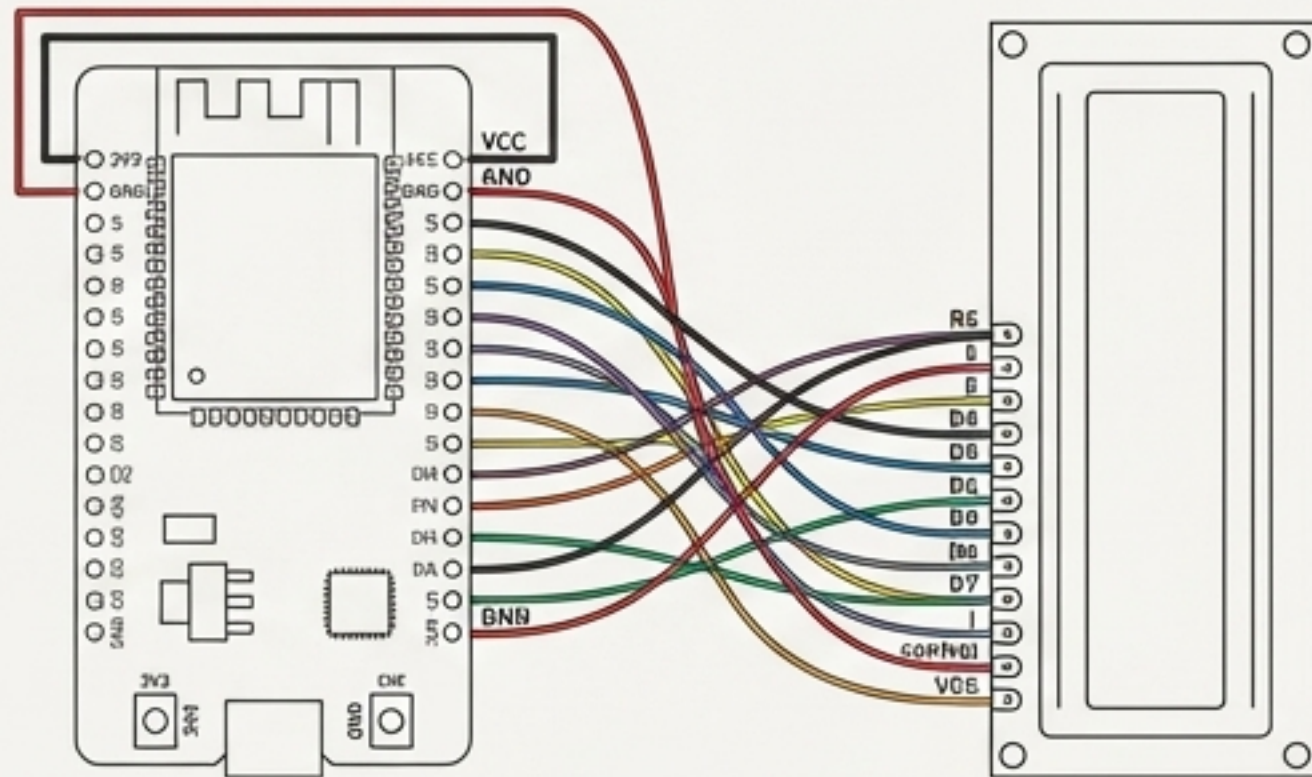
หลักการทำงานของจอ LCD



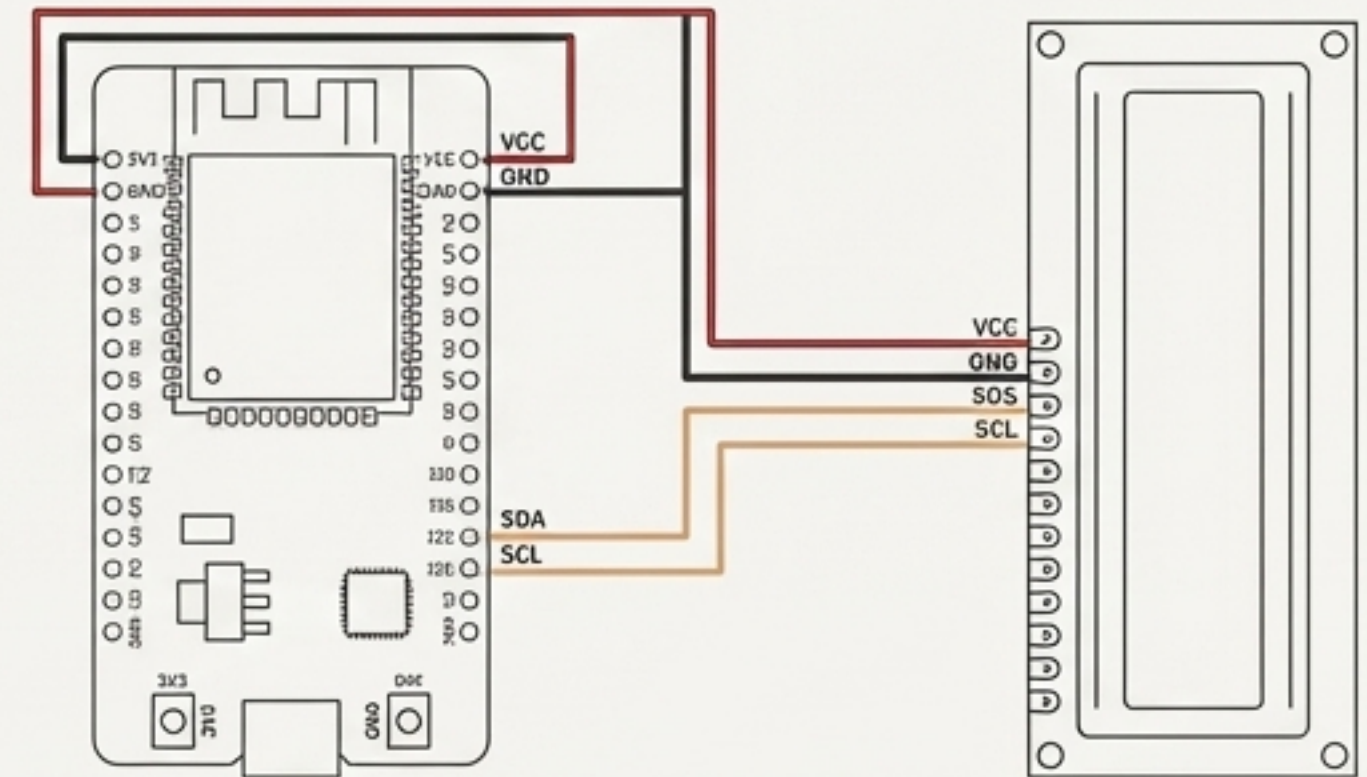
ไลบรารี `LiquidCrystal` และ `LiquidCrystal_I2C` ถูกสร้างขึ้นมาเพื่อสั่งงานชั้น Liquid Crystal นี้เอง

ลดความซับซ้อน: จาก 10+ สู่ 2 เส้น

การเชื่อมต่อแบบ Parallel (ดั้งเดิม)

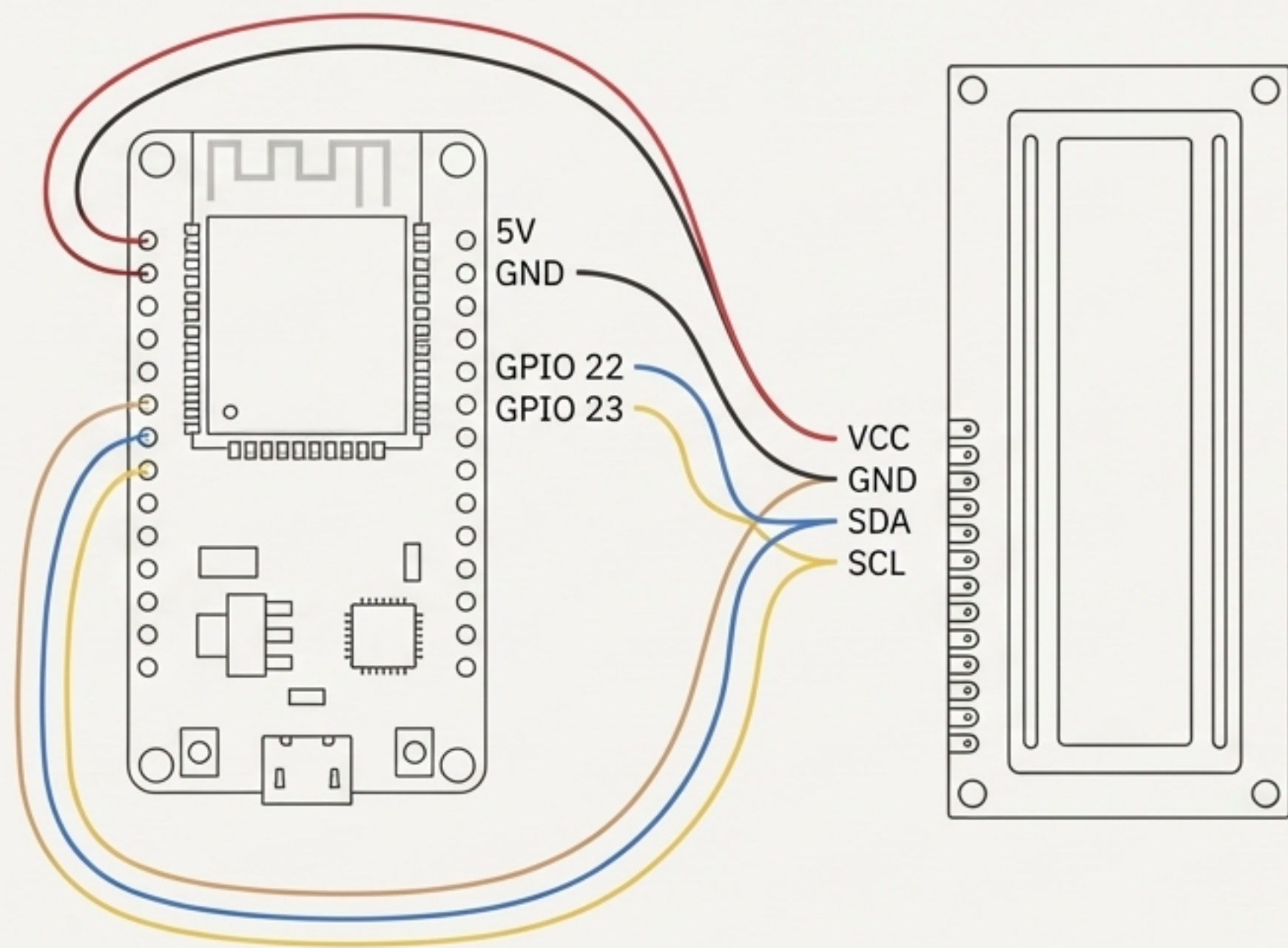


การเชื่อมต่อแบบ I2C



`LiquidCrystal_I2C` คือไลบรารีที่ช่วยให้เราควบคุมจอ LCD ผ่านการสื่อสารแบบ I2C ทำให้ประหยัดขา GPIO ของ ESP32 ไปได้อย่างมหาศาล

การเชื่อมต่อ ESP32 กับจอ LCD I2C

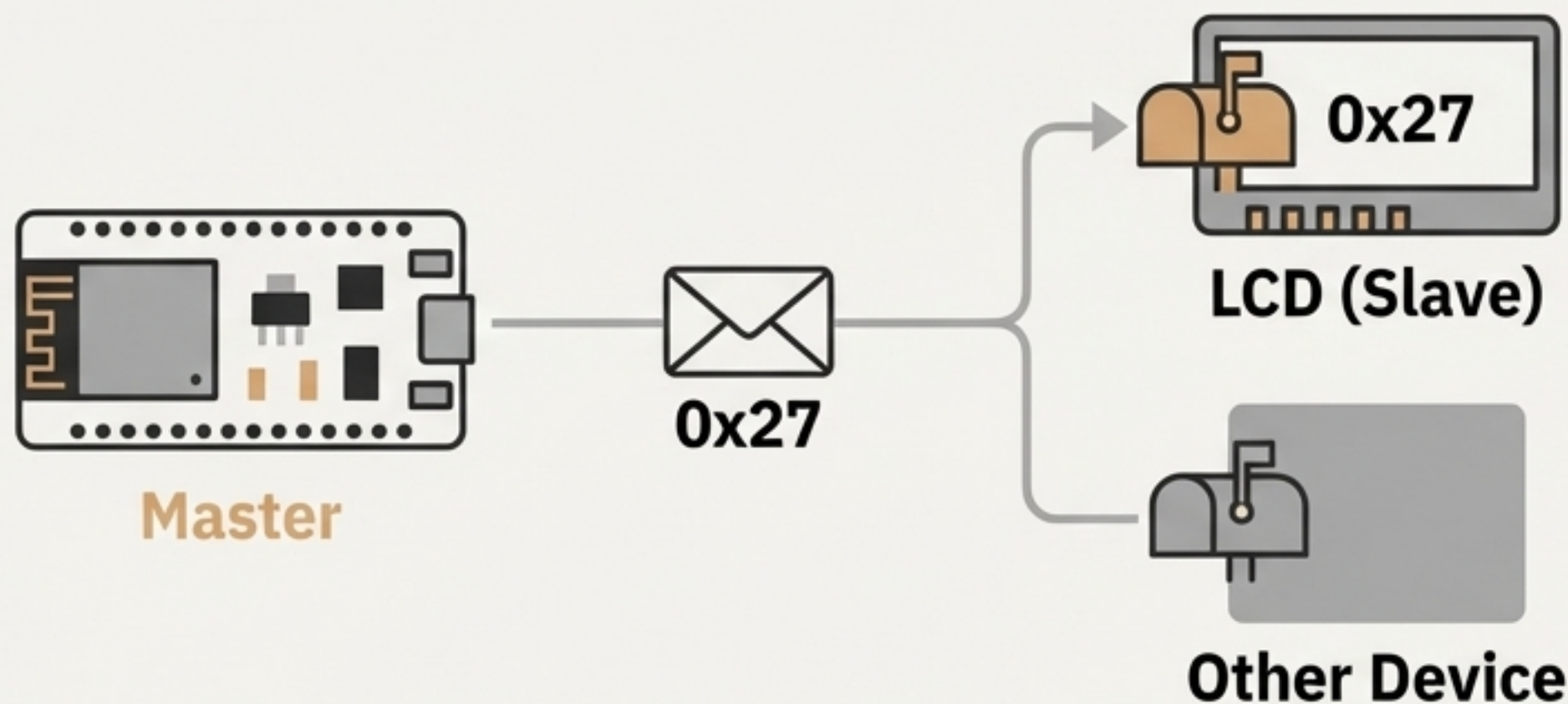


LCD I2C Module	ESP32 Board
VCC	5V หรือ 3.3V (5V is recommended)
GND	GND
SDA	GPIO 22
SCL	GPIO 23



ESP32 สามารถกำหนดขา SDA/SCL ได้อย่างอิสระ ไม่เหมือนบอร์ด Arduino รุ่นเก่า โค้ดของเราจะใช้ GPIO 22 และ 23 เป็นมาตรฐาน

ทุกอุปกรณ์ I2C ต้องมี "ที่อยู่" ของตัวเอง



การสื่อสารแบบ I2C
เปรียบเหมือนการส่งจดหมาย
ESP32 (Master) ต้องรู้ "ที่อยู่"
(Address) ของจอ LCD (Slave)
เพื่อส่งข้อมูลไปให้ถูกตัว

Common Addresses

0x27

0x3F

ไม่แน่ใจว่า Address คืออะไร?

ใช้โค้ด "I2C Scanner" เพื่อค้นหา Address ของอุปกรณ์ที่เชื่อมต่ออยู่โดยอัตโนมัติ

⚠ **คำเตือน:** Address ในโค้ดต้องตรงกับ Address ของฮาร์ดแวร์เป๊ะๆ มิฉะนั้นจะไม่มีแสดงผล

โครงสร้างโค้ด: ส่วนหัวและการประกาศค่า

```
#include <Wire.h>
```

จัดการการสื่อสารแบบ I2C ทั้งหมด

```
#include <LiquidCrystal_I2C.h>
```

รวมคำสั่งสำหรับควบคุมจอ LCD

```
LiquidCrystal_I2C lcd(0x27, 16, 2);
```

I2C Address ของจอ

จำนวนคอลัมน์

จำนวนแถว

ปลูกจอให้พร้อมทำงานใน `void setup()`

```
void setup() {  
  Serial.begin(115200); // 1  
  Wire.begin(22, 23); // 2  
  lcd.begin(16, 2); // 3  
  lcd.backlight(); // 4  
  lcd.print("Hello, ESP32!");  
}
```

- 1** `Serial.begin(115200);`
- เริ่มการสื่อสาร Serial Monitor สำหรับช่วยดีบั๊ก
- 2** `Wire.begin(22, 23);`
- บอก ESP32 ให้เริ่ม I2C โดยใช้ **SDA=GPIO22** และ **SCL=GPIO23**
- 3** `lcd.begin(16, 2);`
- เริ่มต้นการทำงานของจอขนาด 16x2
- 4** `lcd.backlight();`
- เปิดไฟพื้นหลังของจอ

การแสดงผลข้อความและข้อมูลแบบไบนามิก



`lcd.setCursor(column, row);`
ย้ายเคอร์เซอร์ไปยังตำแหน่งที่ต้องการ

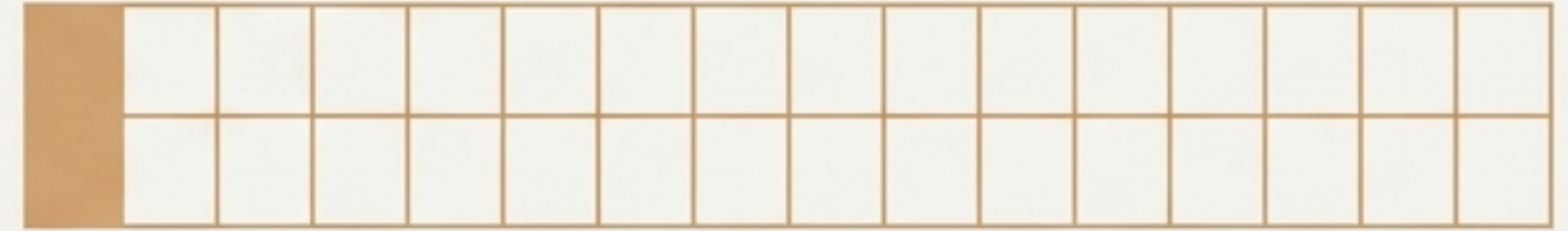


`lcd.print("Message");`
แสดงข้อความหรือค่าของตัวแปร



`lcd.clear();`
ล้างข้อมูลทั้งหมดบนหน้าจอ

(0, 0)



(0, 1)

```
// แสดงเวลาที่ ESP32 ทำงาน
lcd.clear(); // ล้างจอก่อน
lcd.setCursor(0, 0);
lcd.print("Uptime:");
lcd.setCursor(0, 1);
lcd.print(millis() / 1000);
lcd.print(" sec");
delay(1000);
```

❗ การใช้ `lcd.clear()` บ่อยๆ ใน `loop()` อาจทำให้จอกะพริบ ควรใช้เท่าที่จำเป็น

โค้ดฉบับสมบูรณ์: พร้อมใช้งาน

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// กำหนด Address และขนาดของจอ
// ที่อยู่ส่วนใหญ่คือ 0x27 หรือ 0x3F
LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup() {
  // เริ่มต้น Serial Monitor
  Serial.begin(115200);

  // เริ่มต้น I2C บนขาที่กำหนด (SDA=22, SCL=23)
  Wire.begin(22, 23);

  // เริ่มต้นจอ LCD และเปิดไฟ Backlight
  lcd.begin(16, 2);
  lcd.backlight();

  // แสดงข้อความต้อนรับ
  lcd.setCursor(0, 0);
  lcd.print("Hello ESP32!");
  lcd.setCursor(0, 1);
  lcd.print("LCD I2C Ready");
}

void loop() {
  // แสดงเวลาที่บอร์ดทำงาน (หน่วย: วินาที)
  lcd.setCursor(0, 0);
  lcd.print("Running Time: ");
  lcd.setCursor(0, 1);
  lcd.print(millis() / 1000);
  lcd.print(" sec "); // มีเว้นวรรคเพื่อลบเลขเท่า
  delay(1000);
}
```


ปัญหาที่พบบ่อยและแนวทางแก้ไข



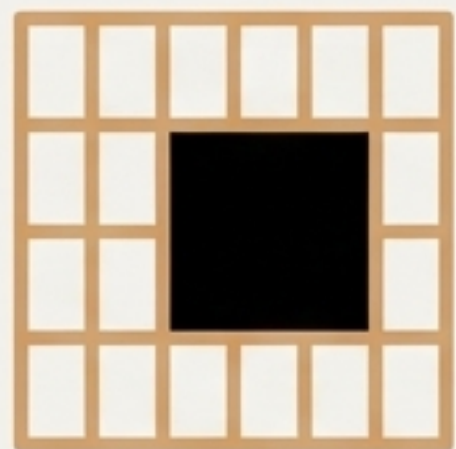
อาการ: จอไม่ติด, ไม่มีไฟ
Backlight

วิธีแก้: ตรวจสอบการเชื่อมต่อสาย
VCC และ GND



อาการ: จอติด แต่ไม่มีตัวอักษรขึ้น

วิธีแก้: Address I2C ในโค้ดไม่ตรงกับของจริง ให้ใช้ I2C Scanner ตรวจสอบ



อาการ: ตัวอักษรแสดงเป็นบล็อก
สีเหลี่ยม หรือจางเกินไป

วิธีแก้: ใช้ไขควงเล็กๆ ปรับตัวต้าน
ทานปรับค่าได้ (Potentiometer)
สีฟ้าด้านหลังโมดูลเพื่อปรับ
Contrast



อาการ: ตัวอักษรกะพริบถี่ๆ

วิธีแก้: ลดความถี่ในการใช้
`lcd.clear()` ในฟังก์ชัน `loop()`

ต่อยอดสู่โปรเจกต์ของคุณ



แสดงค่าจากเซ็นเซอร์

- อ่านค่าอุณหภูมิและความชื้นจาก DHT22
- แสดงค่าความเข้มแสงจาก LDR



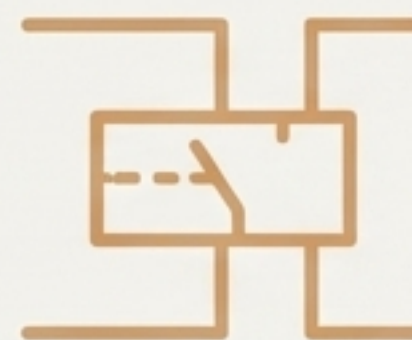
สร้างเมนูควบคุม

- ใช้ปุ่มกดเพื่อสลับหน้าจอแสดงผลข้อมูล
- สร้างเมนูสำหรับตั้งค่าต่างๆ



แสดงสถานะ IoT

- แสดงสถานะการเชื่อมต่อ Wi-Fi
- แจ้งเตือนเมื่อได้รับข้อมูลจากอินเทอร์เน็ต



สร้างแผงควบคุมรีเลย์

- แสดงสถานะเปิด/ปิดของรีเลย์
- ใช้เป็นหน้าจอสำหรับควบคุมอุปกรณ์ไฟฟ้า

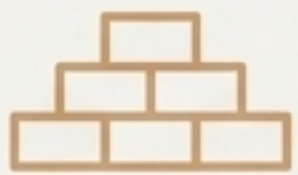
จากหลักการสู่การลงมือทำ: บทสรุป



- **เข้าใจที่มา (You Understand the "Why"):** ชื่อ "Liquid Crystal" มาจากเทคโนโลยีหัวใจสำคัญของจอ ที่ใช้สาร "กึ่งของเหลว-กึ่งผลึก" ควบคุมแสงด้วยไฟฟ้า



- **ง่ายกว่าที่คิด (It's Simpler Than You Thought):** การสื่อสารแบบ I2C ลดการใช้สายไฟเหลือเพียง 2 เส้น (SDA/SCL) ทำให้การเชื่อมต่อกับ ESP32 สะดวกและเป็นระเบียบ



- **พื้นฐานสำคัญ (It's a Fundamental Skill):** การใช้งานจอ LCD คือทักษะพื้นฐานที่จำเป็นสำหรับโปรเจกต์ Microcontroller และ IoT แทบทุกชนิด

ตอนนี้คุณไม่ใช่แค่รู้วิธีใช้ แต่คุณเข้าใจว่ามันทำงานอย่างไร